In [1]:
```
# imports
%pip install seaborn -qq
%pip install grad-cam -qq

import medmnist
import os
import torch
import numpy as np
import torch.nn as nn
import seaborn as sns
import matplotlib.pyplot as plt
from medmnist import BloodMNIST
import torchvision.models as models
from torch.utils.data import DataLoader
import torchvision.transforms as transforms
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

# check if we have CUDA acceleration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"using: {device}")
```

```
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
using: cuda
```

Transforms

In [2]:
```
"""
Image transformation pipelines for model training and evaluation.
- train_transforms: Applies random augmentations (flips, rotation, color jitter) to
- test_transforms: Standardizes validation/test data (tensor conversion, ImageNet n
"""

# image data augmentation for training
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(25),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # I
])

# no augmentation for val or test
test_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # I
])
```

Load Datasets

In [3]:
```
"""
Directory configuration and dataset initialization.
- Sets file paths for the BloodMNIST data and output figures.
- Loads the train, validation, and test splits, applying the appropriate transforma
"""

# change the data_folder path if needed
data_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/data"
figures_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/figures"
models_folder = "/blue/bme6938/gael.garcia/bme6938-project-2/models"

# reload the data with the transforms applied
train_dataset = BloodMNIST(split='train', download=False, size=224, root=data_folde
val_dataset   = BloodMNIST(split='val',   download=False, size=224, root=data_folde
test_dataset  = BloodMNIST(split='test',  download=False, size=224, root=data_folde

print(f"Training dataset length: {len(train_dataset)}")
print(f"Training dataset length: {len(val_dataset)}")
print(f"Training dataset length: {len(test_dataset)}")
```

```
Training dataset length: 11959
Training dataset length: 1712
Training dataset length: 3421
```

DataLoaders

In [4]:
```
"""
Data iterators to handle batching during the training loop.
- train_loader: Batches the data (size 64) and shuffles to ensure random sampling d
- val_loader & test_loader: Batches the data sequentially (no shuffling) for consis
"""

# define our dataloaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False) # note that we D
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Model 1: Custom CNN (from scratch)

In [5]:
```
"""
Custom Convolutional Neural Network (CNN) architecture.
- Features: 4 sequential convolutional blocks (Conv2d, BatchNorm, ReLU, MaxPool) to
- Classifier: Uses global average pooling, dense layers, and dropout to map the ext
"""

# create a custom CNN from scratch
class CustomCNN(nn.Module):
```

```python
    def __init__(self, num_classes=8):
        super(CustomCNN, self).__init__()

        # extract image features
        # sequential allows us to define multiple layers sequentially (one after th
        self.features = nn.Sequential(

            # block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

            # block 4
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),                          # activation function
            nn.MaxPool2d(2, 2),

        )

        # preform classification
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1,1)),
            nn.Flatten(),                       # convert to 1D vector
            nn.Linear(256, 128),
            nn.ReLU(),                          # activation function
            nn.Dropout(0.5),                    # dropout helps prevent overfitting
            nn.Linear(128, num_classes),        # want the output to be limited to n
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = CustomCNN().to(device)
print(model)
```

```
CustomCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=
True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (12): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats
=True)
    (14): ReLU()
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): AdaptiveAvgPool2d(output_size=(1, 1))
    (1): Flatten(start_dim=1, end_dim=-1)
    (2): Linear(in_features=256, out_features=128, bias=True)
    (3): ReLU()
    (4): Dropout(p=0.5, inplace=False)
    (5): Linear(in_features=128, out_features=8, bias=True)
  )
)
```

Sanity Check

In [19]:

```
"""
Sanity Check: Pass a dummy batch through the model to verify output dimensions
Input: [4, 3, 64, 64] -> (Batch Size, RGB Channels, Height, Width)
Expected Output: [4, 8] -> (Batch Size, Number of Classes)
"""

with torch.no_grad():
    dummy = torch.rand(4, 3, 224, 224).to(device)
    out = model(dummy)
    print(f"Shape: {out.shape}")
```

```
Shape: torch.Size([4, 8])
```

Loss + Optimizer

In [7]:

```
"""
Training configuration setup.
- criterion: Uses Cross-Entropy to calculate the error for our multi-class classifi
```

```
- optimizer: Uses Adam to update the model's weights based on the computed gradient
- scheduler: Dynamically drops the learning rate if the validation loss plateaus to
"""

criterion = nn.CrossEntropyLoss() # this is our loss function that penalizes the mo
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3) # Adam is the adpative op

# learning rate scheduler: reduces the LR when validation loss stops improving.
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patie
```

Training Loop

In [8]:

```
"""
Complete training loop for the PyTorch model.
- Handles both training (gradient updates) and validation (evaluation) phases per e
- Implements early stopping to prevent overfitting and saves the best model weights
- Tracks and returns loss and accuracy metrics for both phases across all epochs.
"""

# create a function used for training a provided model
def train_model(model, train_loader, val_loader, model_name, num_epochs=50, patienc
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', p

    best_val_loss = float('inf')
    epochs_no_improve = 0
    train_losses, val_losses, train_accs, val_accs = [], [], [], [] # track metrics

    for epoch in range(num_epochs):
        # --- training ---
        model.train()                                       # set model to training
        running_loss, correct, total = 0, 0, 0
        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.squeeze().long().to(device)

            optimizer.zero_grad()                           # clear old gradients f
            outputs = model(images)                         # forward pass
            loss = criterion(outputs, labels)               # compute error
            loss.backward()                                 # compute gradients (ba
            optimizer.step()                                # update model weights

            running_loss += loss.item()
            _, predicted = outputs.max(1)                   # get the index of the
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)

        train_loss = running_loss / len(train_loader)
        train_acc  = correct / total

        # --- validation ---
        model.eval()                                        # set model to evaluati
        val_loss, correct, total = 0, 0, 0
```

```
        with torch.no_grad():                                # disable gradient calc
            for images, labels in val_loader:
                images = images.to(device)
                labels = labels.squeeze().long().to(device)

                outputs = model(images)
                loss = criterion(outputs, labels)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                correct += predicted.eq(labels).sum().item()
                total += labels.size(0)

        val_loss = val_loss / len(val_loader)
        val_acc  = correct / total

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)
        scheduler.step(val_loss)                             # adjust learning rate

        # --- early stopping + save best ---
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            torch.save(model.state_dict(), os.path.join(models_folder, f"best_{mode
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:                        # stop early i
                print(f"early stopping triggered at epoch {epoch+1}")
                break

        print(f"[{model_name}] epoch {epoch+1:02d}/{num_epochs} | train loss: {trai

    print(f"{model_name} done!")
    return train_losses, val_losses, train_accs, val_accs
```

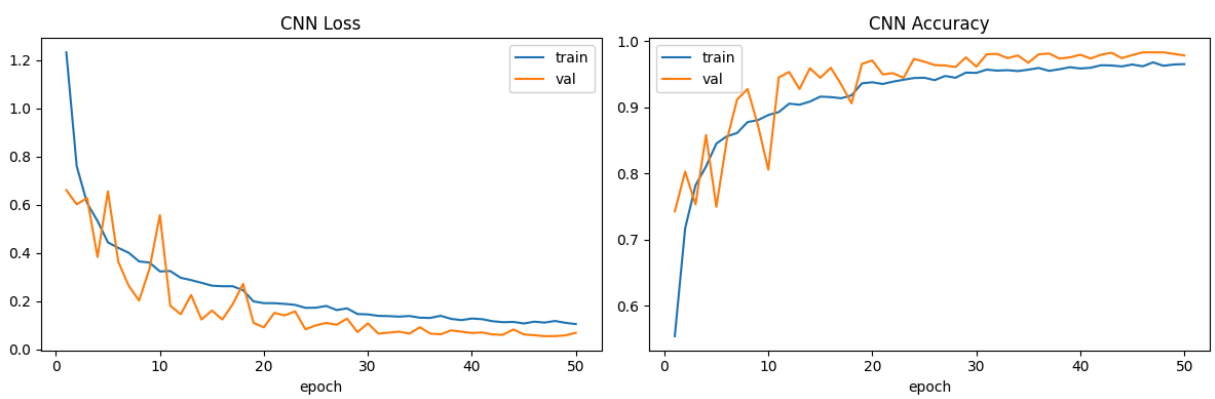
In [9]:

```
print("=== training scratch CNN ===")

# training for max 50 epochs,, 50 was defined in the function, no need to include i
cnn_results = train_model(model, train_loader, val_loader, "cnn")
```

```
=== training scratch CNN ===
[cnn] epoch 01/50 | train loss: 1.2325 acc: 0.5537 | val loss: 0.6605 acc: 0.7424
[cnn] epoch 02/50 | train loss: 0.7600 acc: 0.7169 | val loss: 0.6015 acc: 0.8026
[cnn] epoch 03/50 | train loss: 0.6067 acc: 0.7821 | val loss: 0.6278 acc: 0.7529
[cnn] epoch 04/50 | train loss: 0.5327 acc: 0.8102 | val loss: 0.3831 acc: 0.8581
[cnn] epoch 05/50 | train loss: 0.4438 acc: 0.8451 | val loss: 0.6558 acc: 0.7494
[cnn] epoch 06/50 | train loss: 0.4208 acc: 0.8560 | val loss: 0.3630 acc: 0.8499
[cnn] epoch 07/50 | train loss: 0.4005 acc: 0.8611 | val loss: 0.2635 acc: 0.9118
[cnn] epoch 08/50 | train loss: 0.3646 acc: 0.8776 | val loss: 0.2020 acc: 0.9276
[cnn] epoch 09/50 | train loss: 0.3600 acc: 0.8805 | val loss: 0.3349 acc: 0.8727
[cnn] epoch 10/50 | train loss: 0.3232 acc: 0.8883 | val loss: 0.5569 acc: 0.8055
[cnn] epoch 11/50 | train loss: 0.3246 acc: 0.8927 | val loss: 0.1809 acc: 0.9451
[cnn] epoch 12/50 | train loss: 0.2967 acc: 0.9054 | val loss: 0.1452 acc: 0.9533
[cnn] epoch 13/50 | train loss: 0.2871 acc: 0.9038 | val loss: 0.2251 acc: 0.9276
[cnn] epoch 14/50 | train loss: 0.2760 acc: 0.9086 | val loss: 0.1235 acc: 0.9591
[cnn] epoch 15/50 | train loss: 0.2640 acc: 0.9161 | val loss: 0.1607 acc: 0.9445
[cnn] epoch 16/50 | train loss: 0.2616 acc: 0.9154 | val loss: 0.1236 acc: 0.9597
[cnn] epoch 17/50 | train loss: 0.2617 acc: 0.9136 | val loss: 0.1867 acc: 0.9346
[cnn] epoch 18/50 | train loss: 0.2452 acc: 0.9180 | val loss: 0.2715 acc: 0.9060
[cnn] epoch 19/50 | train loss: 0.1986 acc: 0.9363 | val loss: 0.1097 acc: 0.9655
[cnn] epoch 20/50 | train loss: 0.1915 acc: 0.9378 | val loss: 0.0910 acc: 0.9708
[cnn] epoch 21/50 | train loss: 0.1912 acc: 0.9352 | val loss: 0.1512 acc: 0.9498
[cnn] epoch 22/50 | train loss: 0.1883 acc: 0.9388 | val loss: 0.1405 acc: 0.9515
[cnn] epoch 23/50 | train loss: 0.1843 acc: 0.9416 | val loss: 0.1574 acc: 0.9445
[cnn] epoch 24/50 | train loss: 0.1720 acc: 0.9442 | val loss: 0.0829 acc: 0.9731
[cnn] epoch 25/50 | train loss: 0.1725 acc: 0.9446 | val loss: 0.0993 acc: 0.9690
[cnn] epoch 26/50 | train loss: 0.1799 acc: 0.9410 | val loss: 0.1092 acc: 0.9638
[cnn] epoch 27/50 | train loss: 0.1628 acc: 0.9473 | val loss: 0.1020 acc: 0.9632
[cnn] epoch 28/50 | train loss: 0.1698 acc: 0.9446 | val loss: 0.1272 acc: 0.9609
[cnn] epoch 29/50 | train loss: 0.1463 acc: 0.9524 | val loss: 0.0714 acc: 0.9755
[cnn] epoch 30/50 | train loss: 0.1448 acc: 0.9520 | val loss: 0.1075 acc: 0.9614
[cnn] epoch 31/50 | train loss: 0.1388 acc: 0.9569 | val loss: 0.0651 acc: 0.9801
[cnn] epoch 32/50 | train loss: 0.1377 acc: 0.9554 | val loss: 0.0689 acc: 0.9807
[cnn] epoch 33/50 | train loss: 0.1354 acc: 0.9561 | val loss: 0.0733 acc: 0.9743
[cnn] epoch 34/50 | train loss: 0.1382 acc: 0.9548 | val loss: 0.0649 acc: 0.9784
[cnn] epoch 35/50 | train loss: 0.1310 acc: 0.9569 | val loss: 0.0915 acc: 0.9673
[cnn] epoch 36/50 | train loss: 0.1300 acc: 0.9594 | val loss: 0.0650 acc: 0.9801
[cnn] epoch 37/50 | train loss: 0.1389 acc: 0.9551 | val loss: 0.0621 acc: 0.9813
[cnn] epoch 38/50 | train loss: 0.1263 acc: 0.9575 | val loss: 0.0785 acc: 0.9737
[cnn] epoch 39/50 | train loss: 0.1211 acc: 0.9606 | val loss: 0.0731 acc: 0.9755
[cnn] epoch 40/50 | train loss: 0.1276 acc: 0.9586 | val loss: 0.0674 acc: 0.9796
[cnn] epoch 41/50 | train loss: 0.1249 acc: 0.9598 | val loss: 0.0698 acc: 0.9737
[cnn] epoch 42/50 | train loss: 0.1163 acc: 0.9635 | val loss: 0.0613 acc: 0.9796
[cnn] epoch 43/50 | train loss: 0.1124 acc: 0.9632 | val loss: 0.0602 acc: 0.9825
[cnn] epoch 44/50 | train loss: 0.1136 acc: 0.9618 | val loss: 0.0823 acc: 0.9743
[cnn] epoch 45/50 | train loss: 0.1070 acc: 0.9647 | val loss: 0.0616 acc: 0.9790
[cnn] epoch 46/50 | train loss: 0.1145 acc: 0.9620 | val loss: 0.0584 acc: 0.9831
[cnn] epoch 47/50 | train loss: 0.1108 acc: 0.9679 | val loss: 0.0544 acc: 0.9831
[cnn] epoch 48/50 | train loss: 0.1175 acc: 0.9628 | val loss: 0.0545 acc: 0.9831
[cnn] epoch 49/50 | train loss: 0.1097 acc: 0.9647 | val loss: 0.0577 acc: 0.9807
[cnn] epoch 50/50 | train loss: 0.1046 acc: 0.9652 | val loss: 0.0683 acc: 0.9784
cnn done!
```

Training Curve Plot

In [18]:
```
"""
Visualization of the training and validation metrics for the CNN trained from scrat
- Unpacks the results from the training loop.
- Generates a side-by-side plot of Loss and Accuracy across epochs.
- Saves the generated figure to the designated figures folder.
"""

# unpack the results returned from the train_model function
cnn_train_losses, cnn_val_losses, cnn_train_accs, cnn_val_accs = cnn_results

epochs_ran = range(1, len(cnn_train_losses) + 1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# plot 1: loss
ax1.plot(epochs_ran, cnn_train_losses, label='train')
ax1.plot(epochs_ran, cnn_val_losses, label='val')
ax1.set_title('CNN Loss')
ax1.set_xlabel('epoch')
ax1.legend()

# plot 2: accuracy
ax2.plot(epochs_ran, cnn_train_accs, label='train')
ax2.plot(epochs_ran, cnn_val_accs, label='val')
ax2.set_title('CNN Accuracy')
ax2.set_xlabel('epoch')
ax2.legend()

plt.tight_layout()
plt.savefig(os.path.join(figures_folder, 'cnn_training_curve.png'), dpi=150)
plt.show()
```



Pretrained Models (ResNet and EfficientNet)

In [11]:
```
"""
Pre-trained model initialization for Transfer Learning.
- Loads ResNet50 and EfficientNet-B0 architectures with standard ImageNet weights.
- Replaces the final classification layers to output our specific 8 classes instead
"""
```

```python
# load pretrained ResNet50 model
def get_resnet50(num_classes=8):
    model = models.resnet50(weights='IMAGENET1K_V1')

    # swap the final fully connected (fc) layer to output 8 classes
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

# load pretrained EfficientNet-B0 model
def get_efficientnet(num_classes=8):
    model = models.efficientnet_b0(weights='IMAGENET1K_V1')

    # swap the final linear layer in the classifier block
    model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_classes)
    return model

resnet_model = get_resnet50().to(device)
efficientnet_model = get_efficientnet().to(device)

print("resnet50 ready")
print("efficientnet_b0 ready")
```

```
resnet50 ready
efficientnet_b0 ready
```

In [12]:

```python
print("\n=== training resnet50 ===")
# training for max 50 epochs,, 50 was defined in the function, no need to include i
resnet_results = train_model(resnet_model, train_loader, val_loader, "resnet50")

print("\n=== training efficientnet ===")
# training for max 50 epochs,, 50 was defined in the function, no need to include i
efficientnet_results = train_model(efficientnet_model, train_loader, val_loader, "e
```

```
=== training resnet50 ===
[resnet50] epoch 01/50 | train loss: 0.3344 acc: 0.8881 | val loss: 1.8177 acc: 0.59
70
[resnet50] epoch 02/50 | train loss: 0.2054 acc: 0.9304 | val loss: 0.3965 acc: 0.85
98
[resnet50] epoch 03/50 | train loss: 0.1475 acc: 0.9489 | val loss: 0.1533 acc: 0.94
33
[resnet50] epoch 04/50 | train loss: 0.1312 acc: 0.9551 | val loss: 0.1403 acc: 0.94
74
[resnet50] epoch 05/50 | train loss: 0.1018 acc: 0.9668 | val loss: 0.1391 acc: 0.95
21
[resnet50] epoch 06/50 | train loss: 0.0914 acc: 0.9677 | val loss: 0.1660 acc: 0.95
39
[resnet50] epoch 07/50 | train loss: 0.0943 acc: 0.9687 | val loss: 0.0890 acc: 0.97
14
[resnet50] epoch 08/50 | train loss: 0.0843 acc: 0.9721 | val loss: 0.0890 acc: 0.96
79
[resnet50] epoch 09/50 | train loss: 0.0758 acc: 0.9749 | val loss: 0.0594 acc: 0.98
07
[resnet50] epoch 10/50 | train loss: 0.0734 acc: 0.9729 | val loss: 0.1135 acc: 0.95
44
[resnet50] epoch 11/50 | train loss: 0.0792 acc: 0.9717 | val loss: 0.1620 acc: 0.94
80
[resnet50] epoch 12/50 | train loss: 0.0598 acc: 0.9791 | val loss: 0.0882 acc: 0.97
31
[resnet50] epoch 13/50 | train loss: 0.0648 acc: 0.9788 | val loss: 0.0936 acc: 0.97
43
[resnet50] epoch 14/50 | train loss: 0.0394 acc: 0.9859 | val loss: 0.0468 acc: 0.98
42
[resnet50] epoch 15/50 | train loss: 0.0368 acc: 0.9884 | val loss: 0.0422 acc: 0.98
36
[resnet50] epoch 16/50 | train loss: 0.0331 acc: 0.9892 | val loss: 0.0550 acc: 0.98
07
[resnet50] epoch 17/50 | train loss: 0.0369 acc: 0.9865 | val loss: 0.0345 acc: 0.98
66
[resnet50] epoch 18/50 | train loss: 0.0313 acc: 0.9896 | val loss: 0.0368 acc: 0.98
66
[resnet50] epoch 19/50 | train loss: 0.0340 acc: 0.9876 | val loss: 0.0830 acc: 0.97
20
[resnet50] epoch 20/50 | train loss: 0.0373 acc: 0.9861 | val loss: 0.0447 acc: 0.98
77
[resnet50] epoch 21/50 | train loss: 0.0405 acc: 0.9870 | val loss: 0.0641 acc: 0.98
01
[resnet50] epoch 22/50 | train loss: 0.0245 acc: 0.9918 | val loss: 0.0259 acc: 0.99
07
[resnet50] epoch 23/50 | train loss: 0.0202 acc: 0.9941 | val loss: 0.0285 acc: 0.98
95
[resnet50] epoch 24/50 | train loss: 0.0185 acc: 0.9941 | val loss: 0.0364 acc: 0.98
71
[resnet50] epoch 25/50 | train loss: 0.0196 acc: 0.9931 | val loss: 0.0247 acc: 0.99
36
[resnet50] epoch 26/50 | train loss: 0.0168 acc: 0.9936 | val loss: 0.0395 acc: 0.98
77
[resnet50] epoch 27/50 | train loss: 0.0175 acc: 0.9944 | val loss: 0.0298 acc: 0.99
18
[resnet50] epoch 28/50 | train loss: 0.0212 acc: 0.9925 | val loss: 0.0406 acc: 0.98
```

```
66
[resnet50] epoch 29/50 | train loss: 0.0181 acc: 0.9941 | val loss: 0.0416 acc: 0.98
60
early stopping triggered at epoch 30
resnet50 done!

=== training efficientnet ===
[efficientnet] epoch 01/50 | train loss: 0.2250 acc: 0.9252 | val loss: 0.0644 acc:
0.9825
[efficientnet] epoch 02/50 | train loss: 0.0896 acc: 0.9704 | val loss: 0.0715 acc:
0.9737
[efficientnet] epoch 03/50 | train loss: 0.0725 acc: 0.9755 | val loss: 0.0423 acc:
0.9836
[efficientnet] epoch 04/50 | train loss: 0.0618 acc: 0.9793 | val loss: 0.0382 acc:
0.9866
[efficientnet] epoch 05/50 | train loss: 0.0545 acc: 0.9824 | val loss: 0.0364 acc:
0.9895
[efficientnet] epoch 06/50 | train loss: 0.0511 acc: 0.9843 | val loss: 0.0305 acc:
0.9895
[efficientnet] epoch 07/50 | train loss: 0.0513 acc: 0.9830 | val loss: 0.0615 acc:
0.9778
[efficientnet] epoch 08/50 | train loss: 0.0499 acc: 0.9824 | val loss: 0.0496 acc:
0.9854
[efficientnet] epoch 09/50 | train loss: 0.0436 acc: 0.9844 | val loss: 0.0359 acc:
0.9883
[efficientnet] epoch 10/50 | train loss: 0.0394 acc: 0.9874 | val loss: 0.0317 acc:
0.9936
early stopping triggered at epoch 11
efficientnet done!
```

Evalutation

In [13]:

```
"""
Model evaluation function for the test dataset.
- Disables gradient tracking to speed up inference and save memory.
- Iterates through the test data to collect predictions, true labels, and class pro
- Outputs a detailed classification report and calculates the multi-class ROC-AUC s
"""

def evaluate_model(model, test_loader, model_name):
    model.eval()                                                    # set to evalu
    all_preds, all_labels, all_probs = [], [], []

    with torch.no_grad():                                           # turn off gra
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.squeeze().long()
            outputs = model(images)                                 # forward pass

            # apply softmax to get probabilities (needed for ROC-AUC)
            probs = torch.softmax(outputs, dim=1).cpu().numpy()
            _, predicted = outputs.max(1)                           # get final cl
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.numpy())
```

```
            all_probs.extend(probs)

    # convert lists to numpy arrays
    all_preds  = np.array(all_preds)
    all_labels = np.array(all_labels)
    all_probs  = np.array(all_probs)

    # classification report
    print(f"\n=== {model_name} test results ===")
    print(classification_report(all_labels, all_preds, digits=4))

    auc = roc_auc_score(all_labels, all_probs, multi_class='ovr')
    print(f"ROC-AUC (macro ovr): {auc:.4f}")

    return all_preds, all_labels, all_probs
```

In [14]:

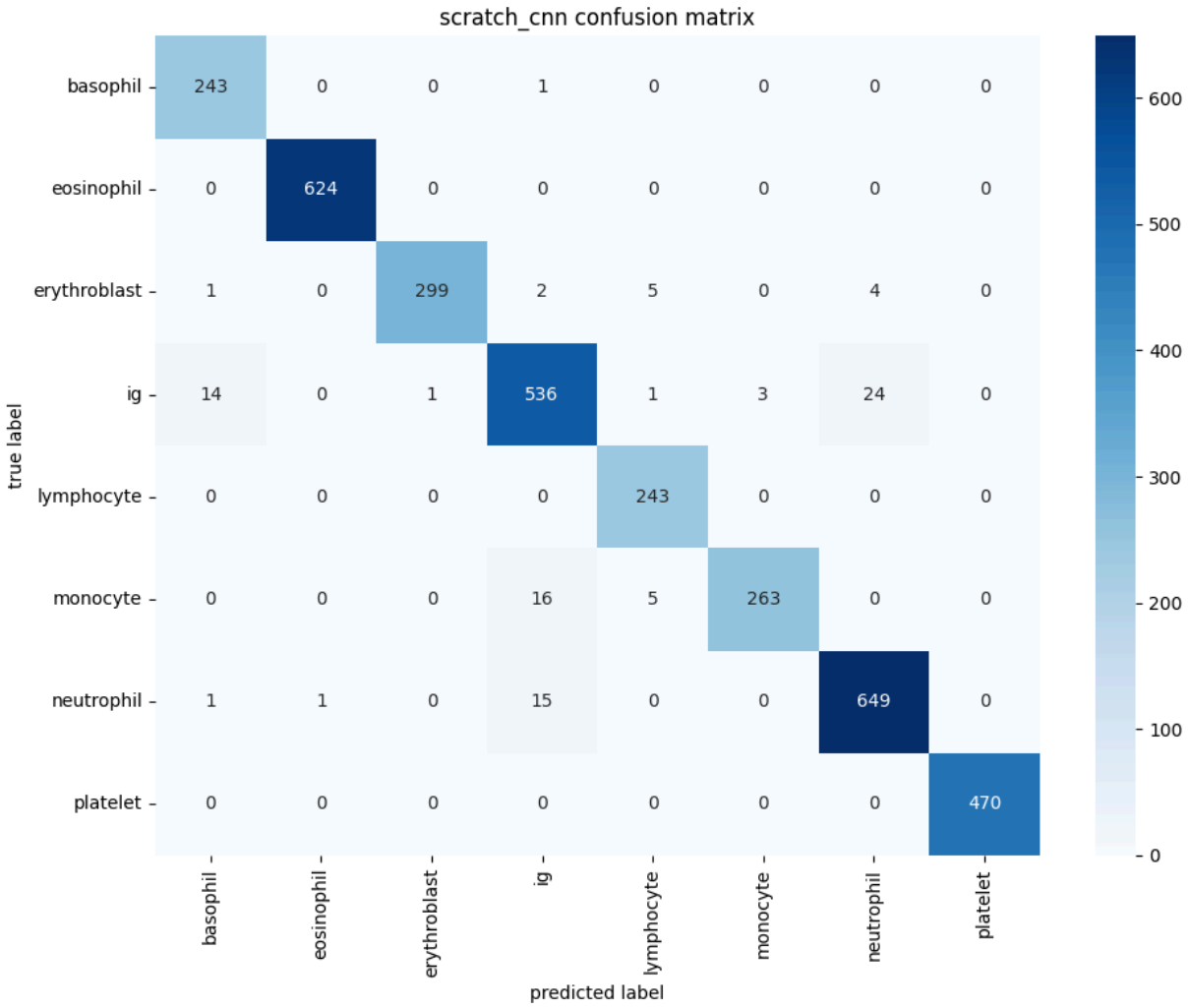
```
# evaluate all 3 models with classification report and ROC/AUC scores
cnn_preds, cnn_labels, cnn_probs           = evaluate_model(model, test_loader, "sc
resnet_preds, resnet_labels, resnet_probs  = evaluate_model(resnet_model, test_load
eff_preds, eff_labels, eff_probs           = evaluate_model(efficientnet_model, tes
```

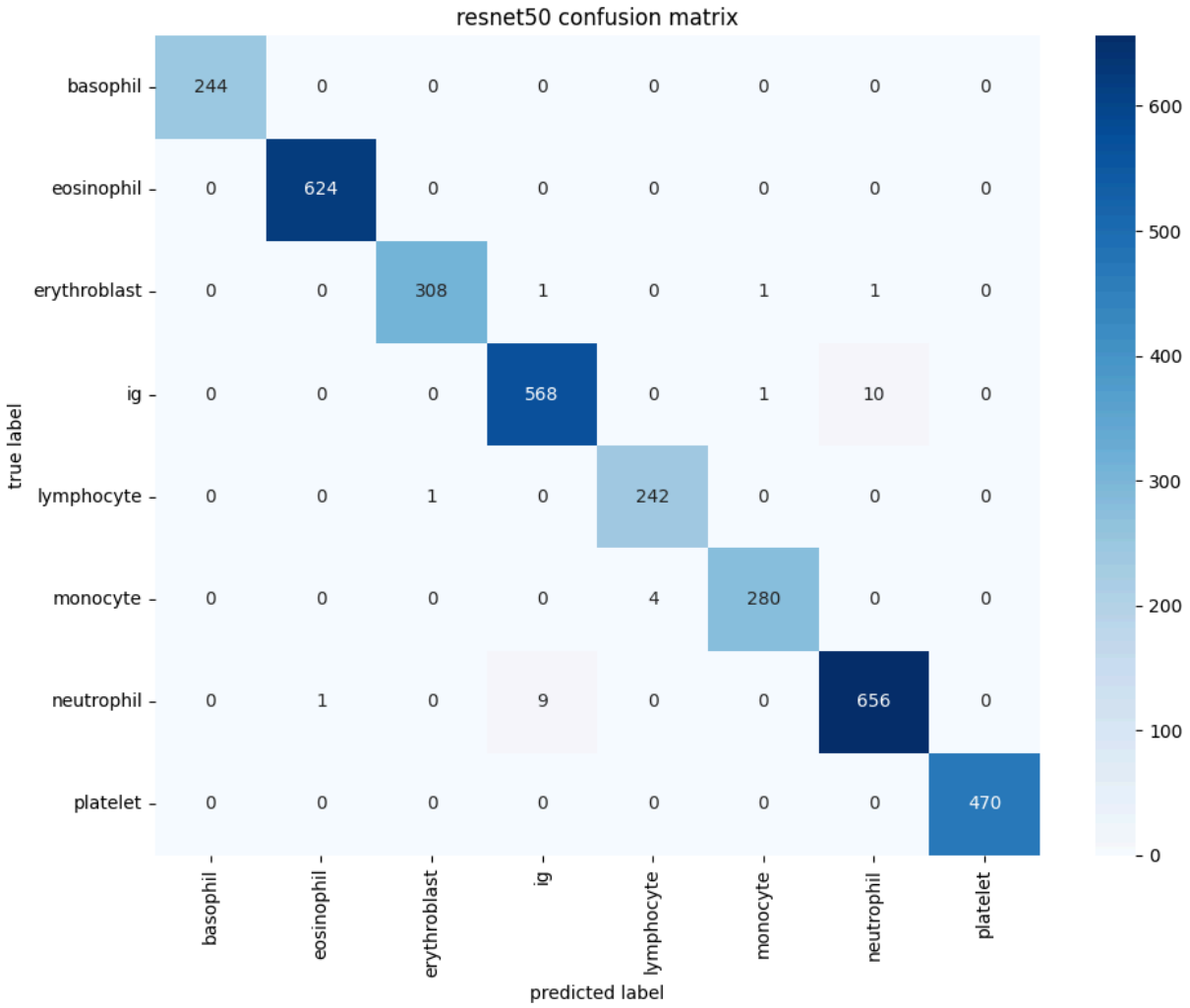
```
=== scratch CNN test results ===
              precision    recall  f1-score   support

           0     0.9382    0.9959    0.9662       244
           1     0.9984    1.0000    0.9992       624
           2     0.9967    0.9614    0.9787       311
           3     0.9404    0.9257    0.9330       579
           4     0.9567    1.0000    0.9779       243
           5     0.9887    0.9261    0.9564       284
           6     0.9586    0.9745    0.9665       666
           7     1.0000    1.0000    1.0000       470

    accuracy                         0.9725      3421
   macro avg     0.9722    0.9729    0.9722      3421
weighted avg     0.9728    0.9725    0.9724      3421

ROC-AUC (macro ovr): 0.9990

=== resnet50 test results ===
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000       244
           1     0.9984    1.0000    0.9992       624
           2     0.9968    0.9904    0.9935       311
           3     0.9827    0.9810    0.9818       579
           4     0.9837    0.9959    0.9898       243
           5     0.9929    0.9859    0.9894       284
           6     0.9835    0.9850    0.9842       666
           7     1.0000    1.0000    1.0000       470

    accuracy                         0.9915      3421
   macro avg     0.9923    0.9923    0.9923      3421
weighted avg     0.9915    0.9915    0.9915      3421

ROC-AUC (macro ovr): 0.9995

=== efficientnet test results ===
              precision    recall  f1-score   support

           0     0.9878    0.9959    0.9918       244
           1     0.9984    1.0000    0.9992       624
           2     0.9810    0.9936    0.9872       311
           3     0.9704    0.9620    0.9662       579
           4     0.9959    0.9918    0.9938       243
           5     0.9463    0.9930    0.9691       284
           6     0.9893    0.9685    0.9788       666
           7     1.0000    0.9979    0.9989       470

    accuracy                         0.9851      3421
   macro avg     0.9836    0.9878    0.9856      3421
weighted avg     0.9853    0.9851    0.9851      3421

ROC-AUC (macro ovr): 0.9993
```

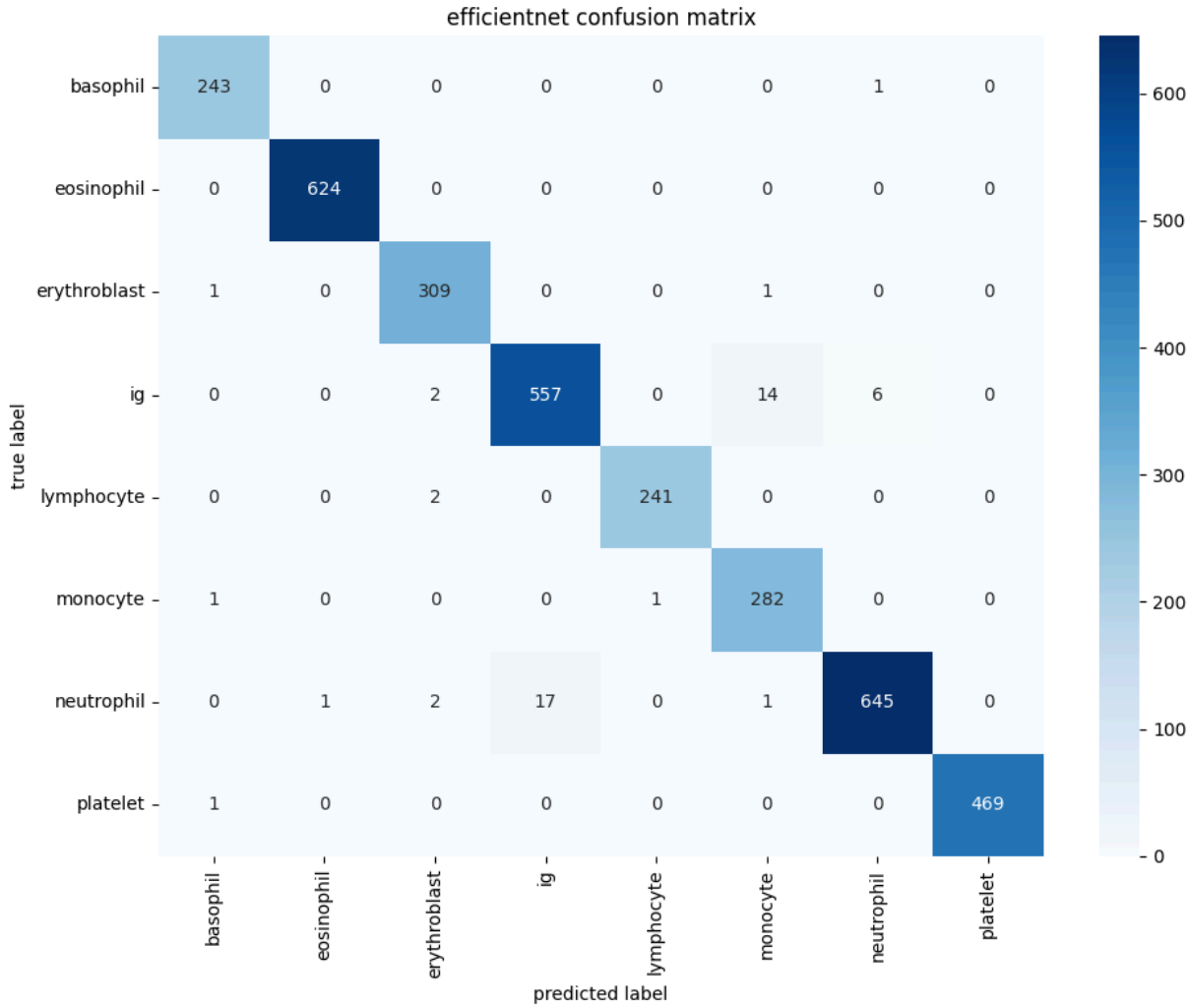
In [15]:

```
"""
Confusion matrix visualization for model comparison.
```

```
- Maps numerical predictions back to their specific blood cell string labels.
- Generates a color-coded heatmap (via seaborn) to easily spot which classes the mo
- Saves each generated matrix to the designated figures folder.
"""

# NOTE: ig = Immature granulocytes (myelocytes, metamyelocytes, and promyelocytes)

cell_types = ['basophil', 'eosinophil', 'erythroblast', 'ig', 'lymphocyte', 'monocy

# define a function to create the confusion matrix for said model
def plot_confusion_matrix(labels, preds, model_name):

    # calculate the raw counts of true vs predicted classifications
    cm = confusion_matrix(labels, preds)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=cell_types, yticklabels=cell_types)
    plt.title(f'{model_name} confusion matrix')
    plt.ylabel('true label')
    plt.xlabel('predicted label')
    plt.tight_layout()
    # save each plot as a png
    plt.savefig(os.path.join(figures_folder, f'confusion_matrix_{model_name}.png'),
    plt.show()

# create the confusion matrix for each of the 3 plots
plot_confusion_matrix(cnn_labels, cnn_preds, 'scratch_cnn')
plot_confusion_matrix(resnet_labels, resnet_preds, 'resnet50')
plot_confusion_matrix(eff_labels, eff_preds, 'efficientnet')
```


scratch_cnn confusion matrix
true label
predicted label
basophil
eosinophil
erythroblast
ig
lymphocyte
monocyte
neutrophil
platelet
243 0 0 1 0 0 0 0
0 624 0 0 0 0 0 0
1 0 299 2 5 0 4 0
14 0 1 536 1 3 24 0
0 0 0 0 243 0 0 0
0 0 0 16 5 263 0 0
1 1 0 15 0 0 649 0
0 0 0 0 0 0 0 470
600
500
400
300
200
100
0

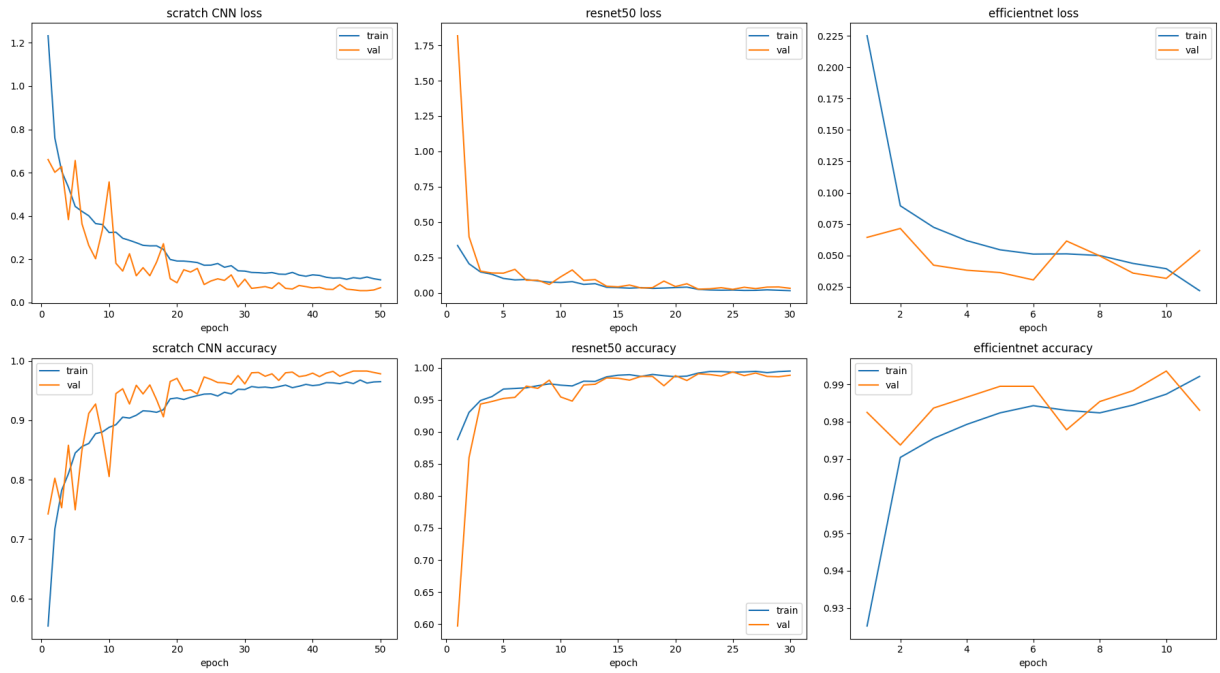

resnet50 confusion matrix
true label
predicted label
basophil
eosinophil
erythroblast
ig
lymphocyte
monocyte
neutrophil
platelet
244 0 0 0 0 0 0 0
0 624 0 0 0 0 0 0
0 0 308 1 0 1 1 0
0 0 0 568 0 1 10 0
0 0 1 0 242 0 0 0
0 0 0 0 4 280 0 0
0 1 0 9 0 0 656 0
0 0 0 0 0 0 0 470
600
500
400
300
200
100
0



In [16]:

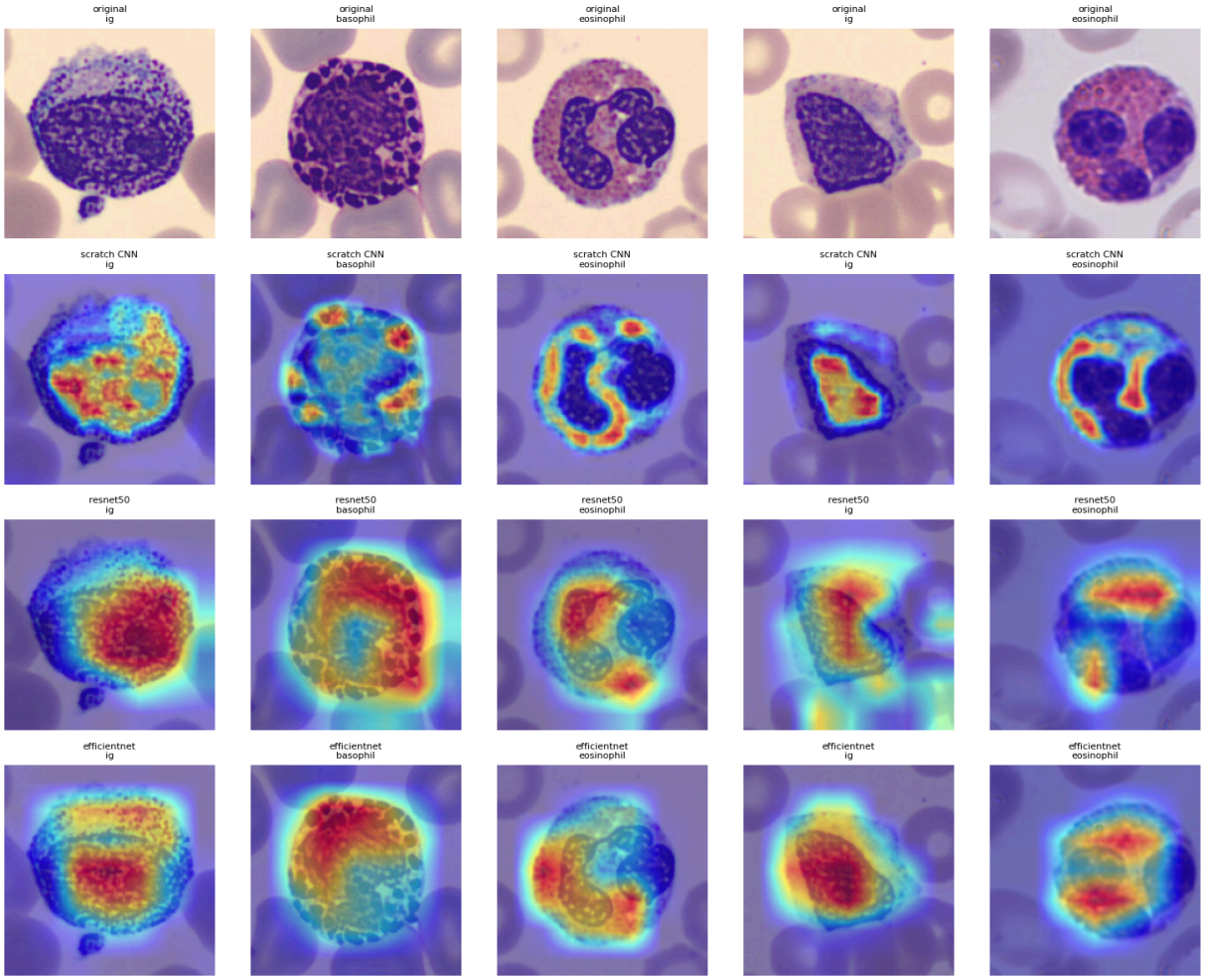
```
"""
Comprehensive training curve visualization.
- Aggregates the results from the Custom CNN, ResNet50, and EfficientNet.
- Generates a 2x3 subplot grid comparing the loss (top row) and accuracy (bottom ro
- Saves the final comparative figure to the designated figures folder.
"""

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# # bundle all results into a dictionary for clean iteration
results = {
    'scratch CNN': cnn_results,
    'resnet50': resnet_results,
    'efficientnet': efficientnet_results
}

for i, (name, (t_loss, v_loss, t_acc, v_acc)) in enumerate(results.items()):
    epochs = range(1, len(t_loss) + 1)

    # top row: loss
    axes[0, i].plot(epochs, t_loss, label='train')
    axes[0, i].plot(epochs, v_loss, label='val')
    axes[0, i].set_title(f'{name} loss')
    axes[0, i].set_xlabel('epoch')
```

```
    axes[0, i].legend()

    # bottom row: accuracy
    axes[1, i].plot(epochs, t_acc, label='train')
    axes[1, i].plot(epochs, v_acc, label='val')
    axes[1, i].set_title(f'{name} accuracy')
    axes[1, i].set_xlabel('epoch')
    axes[1, i].legend()

plt.tight_layout()
plt.savefig(os.path.join(figures_folder, f'all_models_training_curves.png'), dpi=15
plt.show()
```



Grad CAM Plots

In [17]:

```
"""
Grad-CAM (Gradient-weighted Class Activation Mapping) visualizations.
- Initializes Grad-CAM for all three models by targeting their final convolutional
- Un-normalizes a sample of test images to display them in standard RGB format.
- Generates a 4x5 grid comparing the original images against the heatmaps produced
"""

# pick target layer for each model
cam_cnn = GradCAM(model=model, target_layers=[model.features[-2]])
cam_resnet = GradCAM(model=resnet_model, target_layers=[resnet_model.layer4[-1]])
cam_efficient = GradCAM(model=efficientnet_model, target_layers=[efficientnet_model

# grab a few test images
sample_images, sample_labels = next(iter(test_loader))
sample_images = sample_images[:5].to(device)

fig, axes = plt.subplots(4, 5, figsize=(15, 12))
```

```
# row 0: original images
for j in range(5):
    img = sample_images[j].cpu().permute(1, 2, 0).numpy()
    img = img * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
    img = np.clip(img, 0, 1)
    axes[0, j].imshow(img)
    axes[0, j].set_title(f'original\n{cell_types[sample_labels[j].item()]}', fontsi
    axes[0, j].axis('off')

# rows 1-3: gradcam for each model
for i, (cam, name) in enumerate([(cam_cnn, 'scratch CNN'), (cam_resnet, 'resnet50')
    grayscale_cams = cam(input_tensor=sample_images)
    for j in range(5):
        img = sample_images[j].cpu().permute(1, 2, 0).numpy()
        img = img * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
        img = np.clip(img, 0, 1).astype(np.float32)
        cam_image = show_cam_on_image(img, grayscale_cams[j], use_rgb=True)
        axes[i+1, j].imshow(cam_image)
        axes[i+1, j].set_title(f'{name}\n{cell_types[sample_labels[j].item()]}', fo
        axes[i+1, j].axis('off')

plt.tight_layout()
plt.savefig(os.path.join(figures_folder, f'gradcam.png'), dpi=150)
plt.show()
```